
CEN330 Parallel Programming

Homework 2 – Report

Nikol Isaku

Problem 1 (40 pts) – Random Restart Hill Climbing

Hill climbing is a local search algorithm that iteratively improves a solution by moving to a better neighbouring point until no improvement is possible. Its main weakness is convergence to a local maximum.

Random restart hill climbing mitigates this by running the algorithm many times from different random starting points and returning the best result found. This is embarrassingly parallel: each restart is completely independent.

Objective Function

The function used for testing:

```
double objectiveFunction(double x, double y) {  
    return 50 + 20*x*y - 3*x*x*y - 2*x*y*y;  
}  
// Expected maximum: ~99.4 at x≈2.23, y≈3.33  
// Domain: minX=0, maxX=10, minY=0, maxY=10, stepSize=0.01
```

Neighbourhood

From any current point (x_0, y_0) , all 8 grid-adjacent neighbours are evaluated (cardinal + diagonal). The neighbour with the highest objective value becomes the new current point, repeating until no neighbour improves the value.

1a – POSIX Threads Implementation

File: `p1a_pthreads.c`

Compile: `gcc -O2 -o p1a_pthreads p1a_pthreads.c -lpthread -lm`

Run: `./p1a_pthreads 10000 4 0 10 0 10 0.01`

Design:

- Each thread receives a share of the total restarts (distributed with round-robin to balance uneven totals).
- Each thread uses its own `rand_r()` seed to avoid data races on the global `rand()` state.
- After completing all local restarts, each thread locks a global mutex and updates the shared best result if its local best exceeds it.
- The main thread joins all worker threads and prints the final answer.

```
pthread_mutex_lock(&g_mutex);
if (localBestVal > g_bestVal) {
    g_bestVal = localBestVal; g_bestX = localBestX; g_bestY = localBestY;
}
pthread_mutex_unlock(&g_mutex);
```

Test output:

```
Best value : 99.382716
Found at   : x = 2.222196, y = 3.333284
```

1b – OpenMP Implementation

File: p1b_openmp.c

Compile: gcc -O2 -fopenmp -o p1b_openmp p1b_openmp.c -lm

Run: ./p1b_openmp 10000 4 0 10 0 10 0.01

Design:

- The total restart loop is divided among threads with `#pragma omp for schedule(dynamic, 64)`. Dynamic scheduling balances load because individual hill-climbing runs vary in length.
- Each thread accumulates a thread-local best. A `#pragma omp critical` block at the end merges local bests into the global best.
- Each thread has its own `rand_r` seed derived from its thread ID and the clock, avoiding contention on `rand()`.

```
#pragma omp parallel
{
    // Each thread: local best over its share of restarts
    #pragma omp for schedule(dynamic, 64)
    for (int i = 0; i < totalRestarts; ++i) { ... }

    #pragma omp critical
    { if (localBestVal > globalBestVal) { ... } }
}
```

1c – MPI Implementation

File: p1c_mpi.c

Compile: mpicc -O2 -o p1c_mpi p1c_mpi.c -lm

Run: mpiexec -n 4 ./p1c_mpi 10000 0 10 0 10 0.01

Design:

- Only rank 0 reads user input. Parameters are broadcast to all processes via `MPI_Bcast`.
- Restarts are divided evenly; any remainder is distributed one per process to the first ranks.
- Each process runs its restarts independently with its own random seed.
- `MPI_Reduce` with `MPI_MAXLOC` identifies the process holding the global maximum value and its rank.
- The winning rank broadcasts its x, y coordinates via `MPI_Bcast`. Rank 0 then prints the result.

```
MPI_Bcast(params, 6, MPI_DOUBLE, 0, MPI_COMM_WORLD);
// ... each process runs its share ...
struct { double val; int rank; } local_mr, global_mr;
MPI_Reduce(&local_mr, &global_mr, 1, MPI_DOUBLE_INT,
           MPI_MAXLOC, 0, MPI_COMM_WORLD);
MPI_Bcast(winner_xy, 2, MPI_DOUBLE, global_mr.rank, MPI_COMM_WORLD);
```

Problem 2 (25 pts) – Bucket Sort with MPI

File: p2_bucketsort.c

Compile: mpicc -O2 -o p2_bucketsort p2_bucketsort.c

Run: mpiexec -n 5 ./p2_bucketsort

Algorithm Overview

Bucket sort divides the value range into buckets, sorts each bucket independently, and concatenates the sorted buckets to obtain the final sorted array.

Steps in our MPI implementation:

- Rank 0 generates 1 000 random integers in [1, 250].
- The value range [1..250] is divided equally among (nprocs - 1) worker processes. Each worker owns a range bucket (e.g., with 4 workers: [1–63], [64–125], [126–188], [189–250]).
- Rank 0 iterates over the input and sends each integer to the appropriate worker using point-to-point MPI_Send.
- Each worker collects its integers, sorts them with the C standard library qsort(), and sends the sorted bucket back to rank 0.
- Rank 0 receives sorted buckets in order and concatenates them. Since each bucket covers a non-overlapping, ascending range, the concatenation is the globally sorted array.
- A verification pass checks that the output is non-decreasing.

Key Design Decisions

- The master (rank 0) does not participate in sorting, keeping the design clean and allowing it to overlap communication.
- Workers send their bucket count first, then the data, so the master knows how many integers to expect from each worker.
- Using (nprocs - 1) sorting workers allows easy scaling from 2 to 100 processes.
- For N = 1 000 integers in [1, 250], many integers share the same value, so bucket sizes vary; qsort handles this correctly.

Problem 3 (30 pts) – Producer-Consumer in OpenMP and MPI

3a – OpenMP Implementation

File: p3a_openmp.c

Compile: gcc -O2 -fopenmp -o p3a_openmp p3a_openmp.c

Run: ./p3a_openmp (producer0.txt, producer1.txt, producer2.txt must exist)

Design:

- 5 threads total: 3 producers (IDs 0–2) and 2 consumers (IDs 3–4).
- A circular buffer of capacity 10 words (BUFFER_SIZE) is shared among all threads.
- Each producer reads one text file and inserts tokens one at a time. If the buffer is full, the producer busy-waits (spin-lock) until a slot is available.
- Consumers spin-wait while the buffer is empty, and exit only when both the buffer is empty and all producers have finished.
- Two OMP locks are used: buf_lock protects all buffer state (head, tail, count, active_producers); print_lock serialises console output to prevent garbled lines.

- The active_producers counter is decremented inside buf_lock when a producer finishes, ensuring consumers see a consistent "done" state.

3b – MPI Implementation

File: p3b_mpi.c

Compile: mpicc -O2 -o p3b_mpi p3b_mpi.c

Run: mpiexec -n 7 ./p3b_mpi (3 producers + 3 consumers + 1 manager)

Design:

- A dedicated manager rank (last rank) owns the shared buffer data structure entirely. This eliminates race conditions by design — only one rank ever reads or writes the buffer.
- Producers (ranks 0..2) send each token to the manager with tag TAG_PRODUCE, then send TAG_DONE when their file is exhausted.
- Consumers (ranks 3..n-2) repeatedly send TAG_CONSUME requests. The manager replies with a TAG_TOKEN message containing the next word, a __WAIT__ sentinel if the buffer is temporarily empty but producers are still running, or TAG_EMPTY when all producers are done and the buffer is drained.
- The manager uses MPI_Probe with MPI_ANY_SOURCE to multiplex incoming messages fairly.

Comparison: OpenMP vs MPI for Producer-Consumer

Metric	OpenMP	MPI
Speed	Faster (direct memory)	Slower (msg per token)
Sync complexity	Moderate (2 locks)	High (5-tag protocol)
Correctness effort	Moderate	Higher (deadlock risk)
Multi-node scaling	Not possible	Scales well
Bottleneck	Buffer lock	Manager rank

Full comparison text is in observations.txt.

Problem 4 (15 pts) – Matrix Multiplication

Problem Statement

Compute the product $C = A \times B$ of two $N \times N$ matrices. This is a classic high-performance computing benchmark because the $O(n^3)$ work is highly parallelisable: every element $C[i][j]$ is independent of all others.

Serial Solution

File: p4_serial.c

The standard triple-loop algorithm:

```
for (int i = 0; i < N; ++i)
  for (int j = 0; j < N; ++j) {
    double sum = 0.0;
    for (int k = 0; k < N; ++k)
      sum += A[i][k] * B[k][j];
```

```
C[i][j] = sum;
}
```

Complexity: $O(N^3)$. For $N = 512$, this takes approximately 0.20 s on a single core.

Shared-Memory Parallel Solution – OpenMP

File: p4_openmp.c

Compile: `gcc -O2 -fopenmp -o p4_openmp p4_openmp.c`

Run: `./p4_openmp 4`

Strategy: Row-stripping. The outermost loop over rows i is distributed across threads. Each thread computes a contiguous block of rows of C . No synchronisation is needed during computation because rows of C are independent.

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; ++i)
    for (int j = 0; j < N; ++j) {
        double sum = 0.0;
        for (int k = 0; k < N; ++k) sum += A[i][k] * B[k][j];
        C[i][j] = sum;
    }
```

A single pragma distributes the work with no extra locking. Static scheduling is appropriate here because all rows require identical work.

Distributed-Memory Parallel Solution – MPI

File: p4_mpi.c

Compile: `mpicc -O2 -o p4_mpi p4_mpi.c -lm`

Run: `mpiexec -n 4 ./p4_mpi`

Strategy: Row-stripping with scatter/gather.

- Rank 0 initialises A and B .
- `MPI_Scatter` distributes `rows_per_proc` rows of A to each process.
- `MPI_Bcast` sends the entire matrix B to all processes. B is needed in full by every process to compute its rows of C .
- Each process computes its stripe of C locally — no inter-process communication during computation.
- `MPI_Gather` collects the computed stripes from all processes at rank 0.
- Rank 0 prints the result and elapsed wall time.

```
MPI_Scatter(A, rows*N, MPI_DOUBLE, localA, rows*N, MPI_DOUBLE, 0, comm);
MPI_Bcast(B, N*N, MPI_DOUBLE, 0, comm);
// ... local computation ...
MPI_Gather(localC, rows*N, MPI_DOUBLE, C, rows*N, MPI_DOUBLE, 0, comm);
```

Note: N must be divisible by `nprocs`. For arbitrary N , `MPI_Scatterv` and `MPI_Gatherv` with per-process counts and displacements would be used.

Performance Summary

Implementation	N = 512	Threads/Procs	C[0][0]
Serial	~0.20 s	1	10739.0 ✓
OpenMP	~0.06 s	4	10739.0 ✓

MPI	~0.07 s	4	10739.0 ✓
-----	---------	---	-----------

Both parallel versions produce identical results to the serial solution. The OpenMP version edges out MPI slightly on a single machine due to lower synchronisation overhead (scatter/gather vs direct memory access).